

# Live Crypto Dashboard

Grupp 4:

Eyoub Beraki Tesfamicael, Hugo Lundberg, Katrin Rylander

# Live Demo av Crypto Dashboard

## Vad vi har byggt:

- System som hämtar live-data från CoinMarketCap API och ExchangeRates API
- Bearbetar data i realtid med Kafka.
- Lagrar information i PostgreSQL.
- Visualiserar allt i en interaktiv dashboard med Streamlit.

# Det som vi är nöjda med

- Användaren kan välja kryptovalutan och fiat valutan som är av intresse och detta uppdaterar hela dashboarden
- Det finns förklaringar om vad olika termer betyder

Application setup using *.env* Environment Variables:

- COINMARKETCAP\_SYMBOLS="*XRP,TRX*"
- EXCHANGE\_RATE\_SYMBOLS="*DKK,NOK,SEK*"

### constants.py

- reads *.env* file
- provides *working default* values
- *prevents hard coded* variables

CoinMarketCapAPI *free*

*updates every 1 min*

ExchangeRatesAPI *free*

*updates every 24h*

### connect\_api.py

- collect data from *all selected crypto*
- read/collect *selected exchange rates*

### coin\_producer.py

- fetches data *every 1 min*
- *splits* collected crypto data
- produce 1 event *for each crypto*

### KAFKA SERVER

- single topic
- 1 message *for each crypto*

### dashboard.py

- *application user interface* in streamlit
- *queries postgresql* every 1 min to update graphs
- also queries postgresql *when user*
  - *filters on* crypto currency
  - *filters on* fiat currency

### POSTGRESQL SERVER

- Read/Write from table: *crypto\_data*
- Read/Write from table: *exchange\_rates*

### coin\_consumer.py

- *transform* raw api data
- appends *exchange rates data*
- writes selected fields *to postgresql*

# Agila arbetssättet

## Kanban:

- **Backlog:** Hanterades via **GitHub Projects**.
- **WIP-limit:** Max **4** uppgifter samtidigt (oftast bara **1** ).
- **Definition of Done:**
  - Koden är skriven, granskad, testad och fungerar enligt kravspecifikationen (som var diskuterad och skissad på tavlan).

## Dagliga stand-ups:

- Började dagen med en planering.
- Avslutade dagen med att planera nästa dag/steg.

## Retro:

- I mitten av projektet (onsdag) diskuterade vi:
  - Vad fungerar? Vad fungerar inte? Hur går vi vidare?
  - Resultat:
    - Mer jämlik fördelning av utrymme för att uttrycka idéer.
    - Mer **mobbprogrammering** istället för parprogrammering, vilket ökade samarbetet.

# Agila arbetssättet: Utmaningar & Lösningar

## Svårt att uppskatta tidsåtgång per issue:

- Vi arbetade **på plats 9-17 varje dag** för att säkerställa tillräcklig tid för utveckling och problemlösning.

## Olika nivåer av kunskap och kompetens:

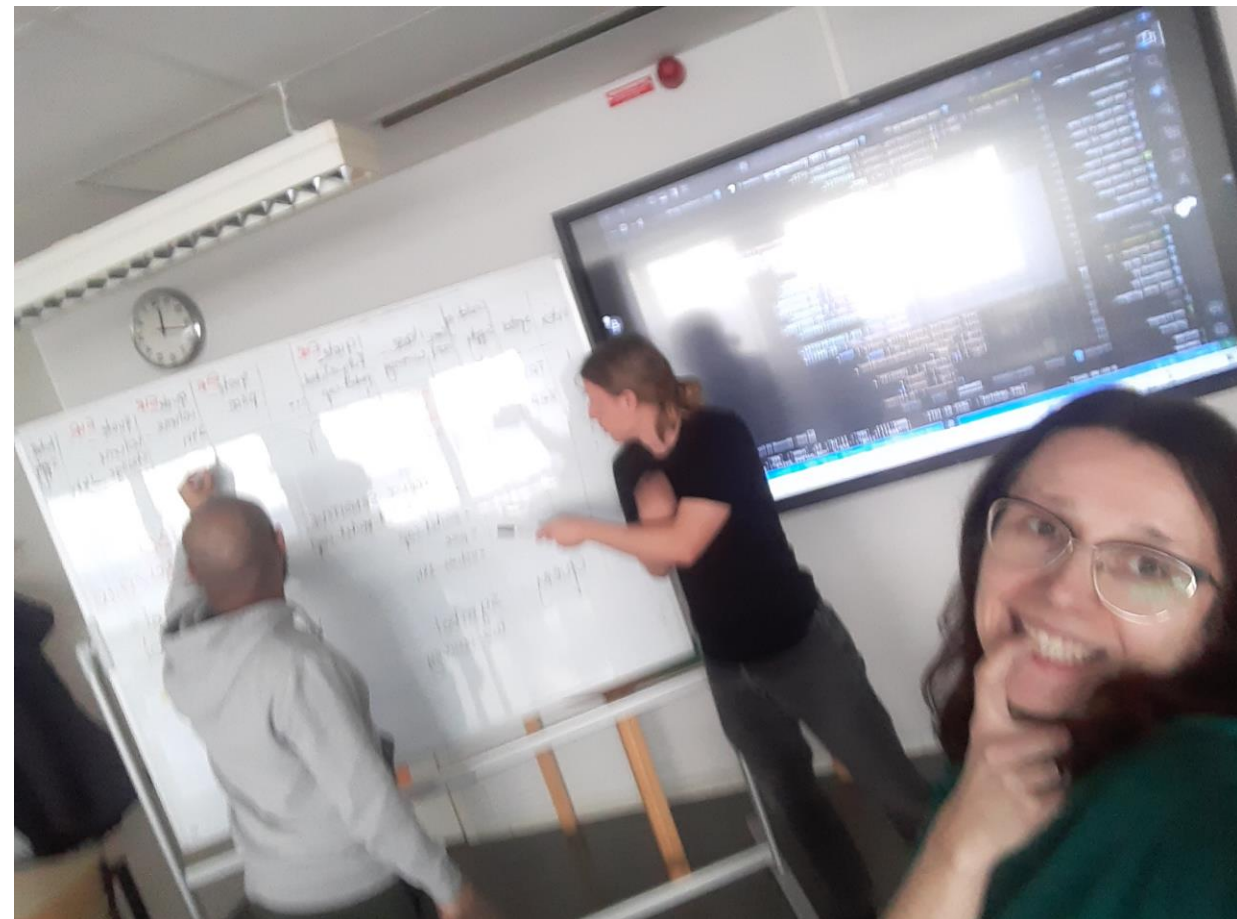
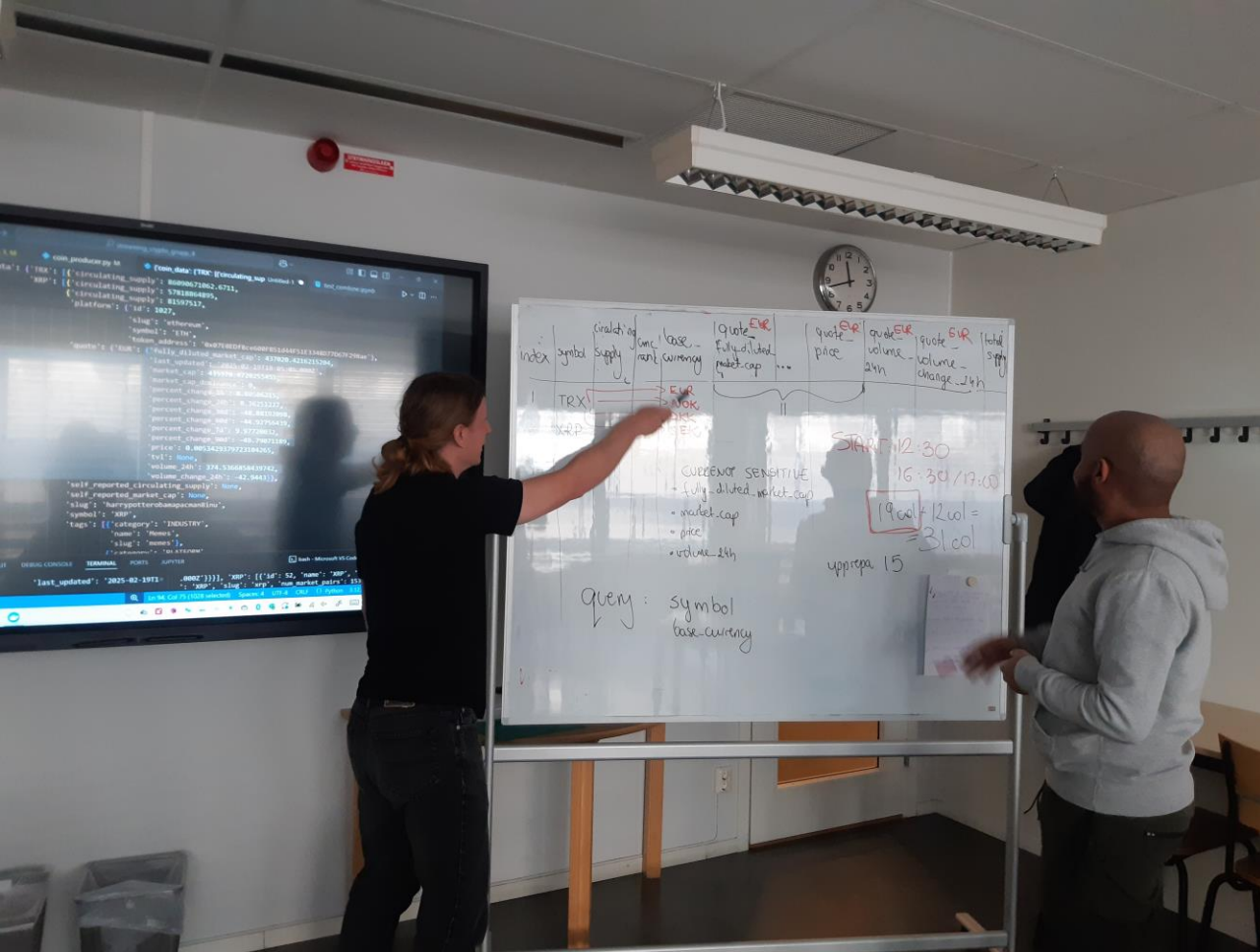
- Använde **whiteboard** för idéutbyte och gemensamt lärande (**onsdag–fredag**).
- Uppmuntrade aktiv kunskapsdelning inom teamet.

## Oväntade buggar:

- Alla testade **lokalt** hur koden fungerade efter varje **pull** för att snabbt identifiera och åtgärda problem.

## Viktig lärdom:

- **Förstå vilken input en funktion kräver innan man designar dess input.** Detta sparade tid och minskade risken för omstrukturering senare.



# Git & Github - Teamarbete

- **Branch-struktur:**
  - main, feature/dashboard, feature/api etc.
- **Pull requests & code reviews**
- **GitHub Issues för buggar & uppgifter.**

## Hur vi undvek git konflikter:

- **Alltid git pull innan git push** för att hålla koden uppdaterad.
- Skapade relevanta **issues** för att hantera buggar systematiskt.
- **Testade lokalt efter varje git pull** för att verifiera att den nya versionen fungerade.
- Regelbundna **code reviews** genom **par- och mobbprogrammering** för att tidigt identifiera problem.



# Förbättringsområden

- **Deployment:** Skapa en **Dockerfile** och anpassa **Docker Compose** för att bygga en **självständig applikation**.
- **Felkontroll:** Implementera **bättre felhantering**, till exempel vid datahämtning från API.
- **Databasstruktur:** Använda **ER-diagram** för att strukturera databasen och separera **statisk** och **dynamisk data** i olika tabeller.
- Hämta fler kryptovalutor samtidigt (till exempel top 10)
- **Notifikationer:** Implementera varningar/notifieringar vid stora prisförändringar.
- **Dashboard:**
  - **Volume/Market Cap Ratio** → Ett mått på likviditet; högre värden indikerar mer aktiv handel.
  - **Liquidity Score (%)** →  $\text{Volume/Market Cap Ratio} \times 100$ , visar likviditet i procent.